Think. What happen to run `n+=1` in Python

Simple Procedure

- Check the object `n` has an `__add__` function.
- The other object (`1`) is passed in to `__add__` function of object `n`.
- `__add__` function can decide how to handle the operation (e.g., `int`, `float`) according to object type, the object may or may not compatible.

  (All processes are conducted dynamically.)

Memory Profiling

메모리할당은 비용이 많이 드는 작업이므로 필요한 양보다 더 많은 메모리를 로컬 풀에 미리 할당해두고 여유롭게 사용하는 방법이 효율적.

- Rewriting your function to work more memory efficiently w/o increasing computation cost.
- Reducing computation cost by using more memory.
- Memory profiler psutil

```
>>> pip3 install psutil
>>> pip3 install memory_profiler
```

Caution!

- Memory profiler make your code 10x ~ 100x slow.
- Overallocation, Garbage collector

→ In python, analyzing memory usage and release processes are not clear.

(Python <u>may not deterministically allocate</u> memory to program.)

Memory Profiling 1 : memory_profiler

Memory profiling of function

```
@profile
def calculate_julia_set(maxiter, zs, cs):
```

(Spent more than 30 minutes …)

```
root@01e7361ded93:/workspace/hpp/w3# python3 -m memory_profiler julia1_memory.py
Filename: julia1_memory.py

Line #    Mem usage    Increment  Occurrences   Line Contents
=============================================================
    20  134.355 MiB  134.355 MiB           1   @profile
    21                                         def calculate_julia_set(maxiter, zs, cs):
    22                                           """Calculate output list using Julia update rule"""
    23  141.832 MiB    7.477 MiB           1     output = [0] * len(zs)
    24  144.926 MiB    0.000 MiB     1000001     for i in range(len(zs)):
    25  144.926 MiB    0.000 MiB     1000000       n = 0
    26  144.926 MiB    0.000 MiB     1000000       z = zs[i]
    27  144.926 MiB    0.000 MiB     1000000       c = cs[i]
    28  144.926 MiB    3.094 MiB    34219980       while abs(z) < 2 and n < maxiter:
    29  144.926 MiB    0.000 MiB    33219980         z = z * z + c
    30  144.926 MiB    0.000 MiB    33219980         n += 1
    31  144.926 MiB    0.000 MiB     1000000       output[i] = n
    32  144.926 MiB    0.000 MiB           1     return output
```

Allocating 1,000*1,000 item

This does not mean that the output list is de finitely 7 MB in size, just that the process gre w by approximately 7 MB during the internal allocation of the list.

Quiz. MiB??

Memory Profiling 1 : memory_profiler

Memory profiling of function

```
@profile
def build_julia_set(desired_width, max_iterations):
```

```
Filename: julia1_memory.py

Line #    Mem usage    Increment  Occurrences   Line Contents
=============================================================
    34   57.543 MiB   57.543 MiB           1   @profile
    35                                         def build_julia_set(desired_width, max_iterations):
    36                                           """Create a list of complex coordinates (zs) and complex parameters (cs), build Julia set"""
    37   57.543 MiB    0.000 MiB           1     x_step = (x2 - x1) / desired_width
    38   57.543 MiB    0.000 MiB           1     y_step = (y1 - y2) / desired_width
    39   57.543 MiB    0.000 MiB           1     x = []
    40   57.543 MiB    0.000 MiB           1     y = []
    41   57.543 MiB    0.000 MiB           1     ycoord = y2
    42   57.543 MiB    0.000 MiB        1001     while ycoord > y1:
    43   57.543 MiB    0.000 MiB        1000       y.append(ycoord)
    44   57.543 MiB    0.000 MiB        1000       ycoord += y_step
    45   57.543 MiB    0.000 MiB           1     xcoord = x1
    46   57.543 MiB    0.000 MiB        1001     while xcoord < x2:
    47   57.543 MiB    0.000 MiB        1000       x.append(xcoord)
    48   57.543 MiB    0.000 MiB        1000       xcoord += x_step
    49   57.543 MiB    0.000 MiB           1     zs = []
    50   57.543 MiB    0.000 MiB           1     cs = []
    51  134.355 MiB   -2.051 MiB        1001     for ycoord in y:
    52  134.355 MiB -2274.145 MiB     1001000       for xcoord in x:
    53  134.355 MiB -2277.469 MiB     1000000         zs.append(complex(xcoord, ycoord))
    54  134.355 MiB -2877.160 MiB     1000000         cs.append(complex(c_real, c_imag))
    55
    56  144.926 MiB  144.926 MiB           1     output = calculate_julia_set(max_iterations, zs, cs)
    57  144.926 MiB    0.000 MiB           1     assert sum(output) == 33219980
    58  144.926 MiB    0.000 MiB           1     return output
```

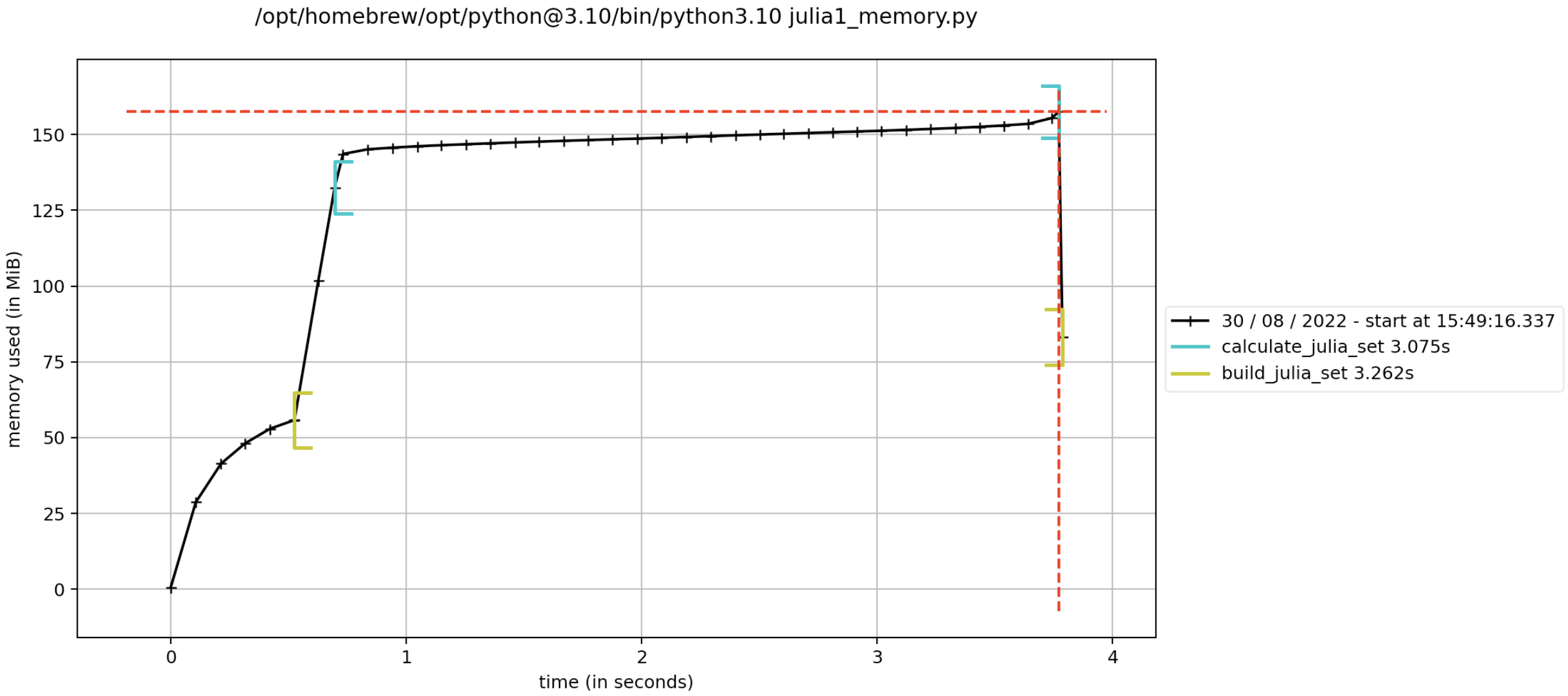
+76.812 MiB

Bug: now the increment column does not match

It is worth that the memory grows by after multiple lines.

Memory Profiling 2 : mprof

```
>>> mprof run julia1_memory.py
>>> mprof plot
```



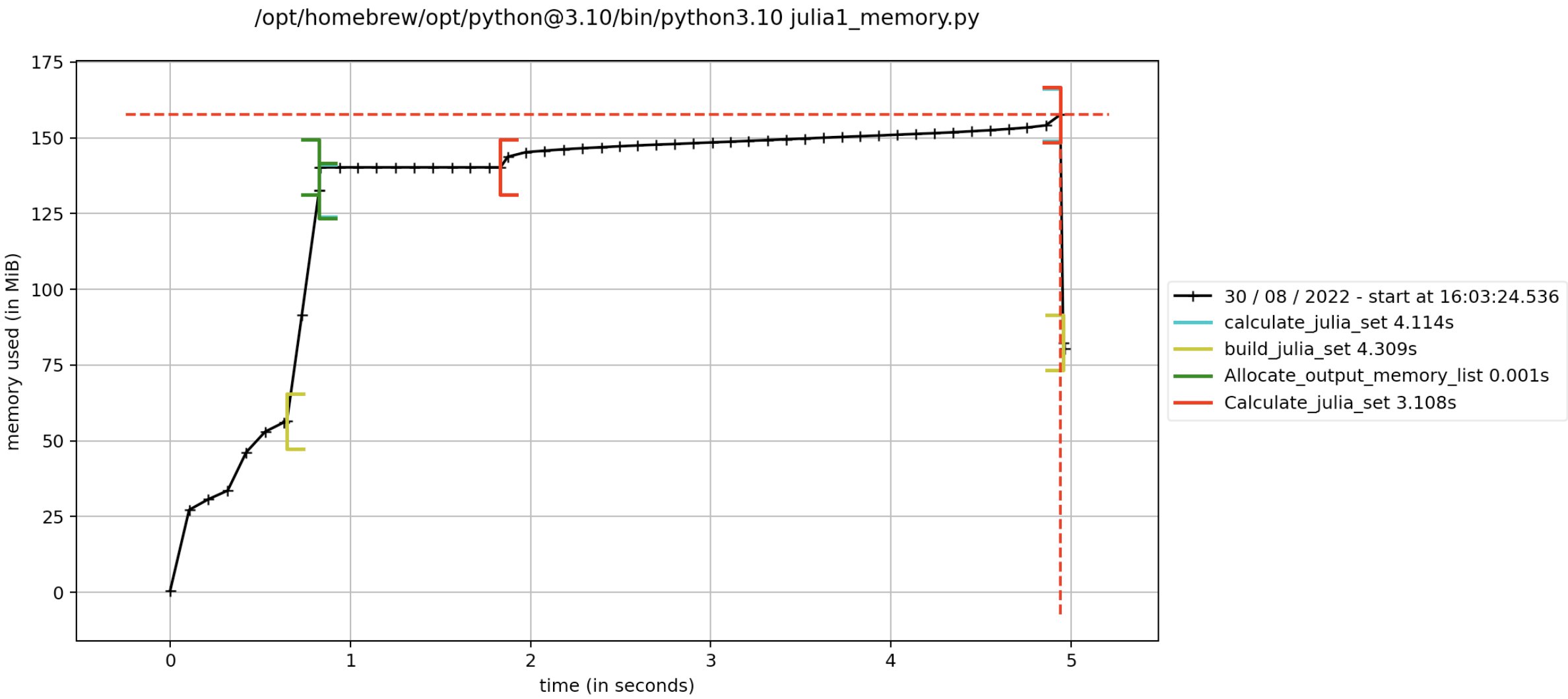
Memory Profiling 2 : mprof

```
>>> mprof run julia1_memory.py
>>> mprof plot
```

```
@profile
def calculate_julia_set(maxiter, zs, cs):
 """Calculate output list using Julia update rule"""
 with profile.timestamp("Allocate_output_memory_list"):
  output = [0] * len(zs)
 time.sleep(1)
 with profile.timestamp("Calculate_julia_set"):
  for i in range(len(zs)):
   n = 0
   z = zs[i]
   c = cs[i]
   while abs(z) < 2 and n < maxiter:
    z = z * z + c
    n += 1
   output[i] = n
 return output
```

Memory Profiling 2 : mprof

```
>>> mprof run julia1_memory.py
>>> mprof plot
```



Check others

- IPython
  - %memit
- pdb debugger
  - --pdb-mmem=000

Bytecode : dis module

```
import dis
dis.dis(build_julia_set)
```

```
y_step = (y1 - y2) / desired_width
```

```
38          12 LOAD_GLOBAL              2 (y1)
            14 LOAD_GLOBAL              3 (y2)
            16 BINARY_SUBTRACT
            18 LOAD_FAST                0 (desired_width)
            20 BINARY_TRUE_DIVIDE
            22 STORE_FAST               3 (y_step)
```

```
while ycoord > y1:
  y.append(ycoord)
  ycoord += y_step
```

```
42          36 SETUP_LOOP              30 (to 68)
      >>    38 LOAD_FAST                6 (ycoord)
            40 LOAD_GLOBAL              2 (y1)
            42 COMPARE_OP               4 (>)
            44 POP_JUMP_IF_FALSE       66

43          46 LOAD_FAST                5 (y)
            48 LOAD_METHOD              4 (append)
            50 LOAD_FAST                6 (ycoord)
            52 CALL_METHOD              1
            54 POP_TOP

44          56 LOAD_FAST                6 (ycoord)
            58 LOAD_FAST                3 (y_step)
            60 INPLACE_ADD
            62 STORE_FAST               6 (ycoord)
            64 JUMP_ABSOLUTE           38
      >>    66 POP_BLOCK
```

Bytecode : dis module

- There are many ways to implement your idea.
- Carefully determine data structure and algorithm

```python
def fn_summation_loop(value=1_000_000):
  total = 0
  for n in range(value):
    total += n
  return total
```

```python
def fn_summation_func(value=1_000_000):
  return sum(range(value))
```

```
10 %timeit fn_summation_loop()
11 %timeit fn_summation_func()
```

```
70.3 ms ± 776 µs per loop (mean ± std. dev. of 7 runs, 10 loops each)
20.6 ms ± 660 µs per loop (mean ± std. dev. of 7 runs, 10 loops each)
```

Bytecode : dis module

```python
def fn_summation_loop(value=1_000_000):
  total = 0
  for n in range(value):
    total += n
  return total
```

```python
def fn_summation_func(value=1_000_000):
  return sum(range(value))
```

```
2           0 LOAD_CONST               1 (0)
            2 STORE_FAST               1 (total)

3           4 SETUP_LOOP              24 (to 30)
            6 LOAD_GLOBAL              0 (range)
            8 LOAD_FAST                0 (value)
           10 CALL_FUNCTION            1
           12 GET_ITER
      >>   14 FOR_ITER                12 (to 28)
           16 STORE_FAST               2 (n)

4          18 LOAD_FAST                1 (total)
           20 LOAD_FAST                2 (n)
           22 INPLACE_ADD
           24 STORE_FAST               1 (total)
           26 JUMP_ABSOLUTE           14
      >>   28 POP_BLOCK

5     >>   30 LOAD_FAST                1 (total)
           32 RETURN_VALUE
```

```
8           0 LOAD_GLOBAL              0 (sum)
            2 LOAD_GLOBAL              1 (range)
            4 LOAD_FAST                0 (value)
            6 CALL_FUNCTION            1
            8 CALL_FUNCTION            1
           10 RETURN_VALUE
```

Conclusion

- Please never avoid profiling based on your intuition.
- Always judge based on measurement results.
- Quick-and-dirty profiling is a good start.
- Our time is more valuable than CPU time.

- <u>In Mobility System</u>
  - CPU Usage and Performance balance (btw. cores/multi-computers/CPU and memory) are directly related to energy consumption and heat problem.
- One more thing
  - Test Driven Development (TDD)